



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees
ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0309 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

Gautham G(192524371)

Sankar R (192525131)

Karthi M E (19251175)

Under the Supervision of

Dr.Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Artificial Intelligence And Data Science
Programme:	B.Tech
Course Code & Course Name	CSA0309 & DATA STRUCTURE
Academic Year / Batch	2026
Faculty Name	Dr Uma Pridarsini P. S
Assignment Title	Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the high-speed security log processing system. CO3: Make use of self-balancing trees (AVL/Red-Black Trees) for ordered, dynamically balanced storage of risk-ranked events. CO4: Make use of hashing, collision-resolution techniques, and priority queues for duplicate detection and high-risk event ranking in C.

	CO5: Develop a robust real-time cyber threat detection engine and evaluate its search performance under varying load factors and data distributions.
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate / L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure; SDG 16 – Peace, Justice and Strong Institutions
Industry / Societal Relevance	Real-time detection of malicious IP addresses, URLs, file hashes and suspicious network activities

B. Assignment Problem / Challenge

Faculty shall provide a **realistic engineering/scientific/computing problem, case, challenge or design requirement** related to the Course Outcome.

The assignment should preferably require students to:

- Apply course concepts rather than reproduce theory.
- Identify and analyze the problem.
- Consider requirements and constraints.
- Develop or compare alternative solutions where appropriate.
- Use relevant modern engineering/computing/design tools.
- Interpret results.
- Make and justify engineering decisions.
- Consider appropriate technical, economic, environmental, societal, ethical, safety or sustainability factors wherever relevant.

C. Problem Statement

Problem / Case / Design Challenge:

[Faculty to provide the detailed assignment problem here.]

D. Requirements and Constraints

Students shall identify or be provided with relevant requirements and constraints.

Examples include:

- Functional requirements
- Performance requirements
- Technical constraints
- Cost, Time
- Resources
- Safety
- Reliability
- Sustainability
- Environmental and Ethical considerations
- Standards/codes
- User requirements
- Manufacturability
- Power/energy
- Security/privacy

Only constraints relevant to the particular course/problem need to be considered.

E. Student Work

1. Problem Understanding and Formulation

Explain:

- What is the problem?
- What are the expected outcomes?
- What information/data is available?
- What assumptions are made?

2. Application of Course Knowledge

Identify and apply the relevant:

- Principles
- Theories
- Mathematical models
- Algorithms
- Engineering concepts
- Scientific concepts
- Design methodologies

Show necessary calculations, derivations or reasoning.

3. Solution / Design / Methodology

Develop the proposed solution. Depending upon the nature of the course, this may include:

- Design
- Algorithm
- Model
- Circuit
- Architecture

- Experiment
- Process
- Prototype
- Simulation
- Case analysis
- Data analysis
- Mathematical solution

Where appropriate, consider **more than one possible solution**.

4. Use of Modern Tools

Use appropriate tools wherever relevant.

Examples:

CSE/IT: Python, MATLAB, TensorFlow, cloud platforms, Git, simulation

tools **ECE/EEE:** MATLAB/Simulink, Cadence, Synopsys, Vivado, Multisim,

LTspice **Mechanical:** ANSYS, SolidWorks, CATIA, MATLAB

Civil: STAAD.Pro, ETABS, AutoCAD, Revit

Biomedical: MATLAB, LabVIEW, signal/image-processing tools

Biotechnology: Bioinformatics/data-analysis/simulation tools

5. Students shall provide appropriate evidence of tool usage and outputs. Results and Validation

Present the results using appropriate:

- Tables
- Graphs

- Simulation outputs
- Experimental observations
- Screenshots
- Performance metrics
- Statistical analysis
- Test results

Validate whether the proposed solution satisfies the stated requirements.

6. Analysis and Engineering Decision

Interpret the results.

Where applicable:

- Compare alternatives.
- Identify advantages and limitations.
- Discuss trade-offs.
- Explain why the final solution was selected.
- Justify the decision using quantitative or qualitative evidence.

7. Broader Considerations

Where relevant to the problem, discuss the impact of the proposed solution with respect to:

- Sustainability
- Environment
- Society
- Safety
- Ethics
- Economics

- Accessibility
- Professional responsibility

4. Conclusion

Summarize:

- Proposed solution
- Major findings
- Achievement of requirements
- Limitations
- Possible improvements

5. Student Reflection

- What did you learn from this assignment beyond what was directly taught in the classroom?
- If you were given additional time/resources, what would you improve and why?

6. References

Use an appropriate citation format and acknowledge all external sources, datasets, standards, software and AI-assisted tools used.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15

Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	C O	PO(s))	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment:

Actual CO Attainment:

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action

Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees

1. Problem Statement

Modern networks generate a large number of security events such as IP addresses, URLs, domain names and file hashes. Detecting malicious indicators quickly is important to prevent cyber attacks. A traditional linear search takes **$O(n)$** time and becomes slow when the threat database contains thousands of records. Therefore, an efficient cyber threat detection engine is required. This project proposes a real-time detection system using **Hashing** for fast threat lookup and a **Self-Balancing AVL Tree** for efficient insertion, deletion and sorted searching.

2. Objective

The main objectives of this project are:

- To develop a real-time cyber threat detection system.
- To use hashing for fast identification of known threats.
- To implement an AVL tree for balanced threat storage.
- To support insertion, searching and deletion of threats.
- To reduce threat detection time.
- To compare the performance of linear search, hashing and AVL trees.
- To provide a simple and efficient cybersecurity solution.

3. Requirements and Environment Used

Hardware Requirements:

- Computer/Laptop
- Minimum 4 GB RAM
- Dual-core processor or above

Software Requirements:

- Python 3.x
- VS Code / IDLE
- Windows/Linux
- Command Prompt/Terminal

Programming Concepts:

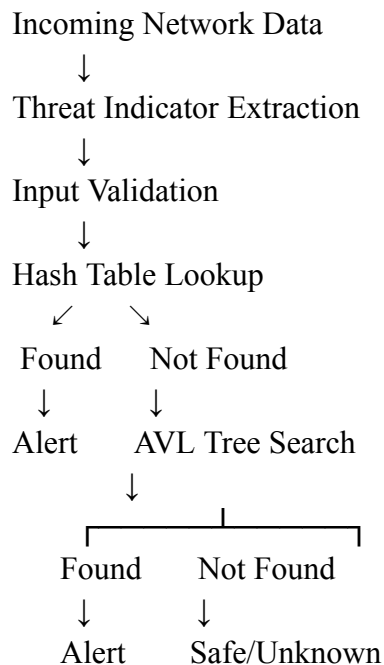
- Hashing
- Hash table
- Binary Search Tree

- AVL Tree
- Tree rotations
- Searching and sorting
- File handling

4. Design / Proposed Solution

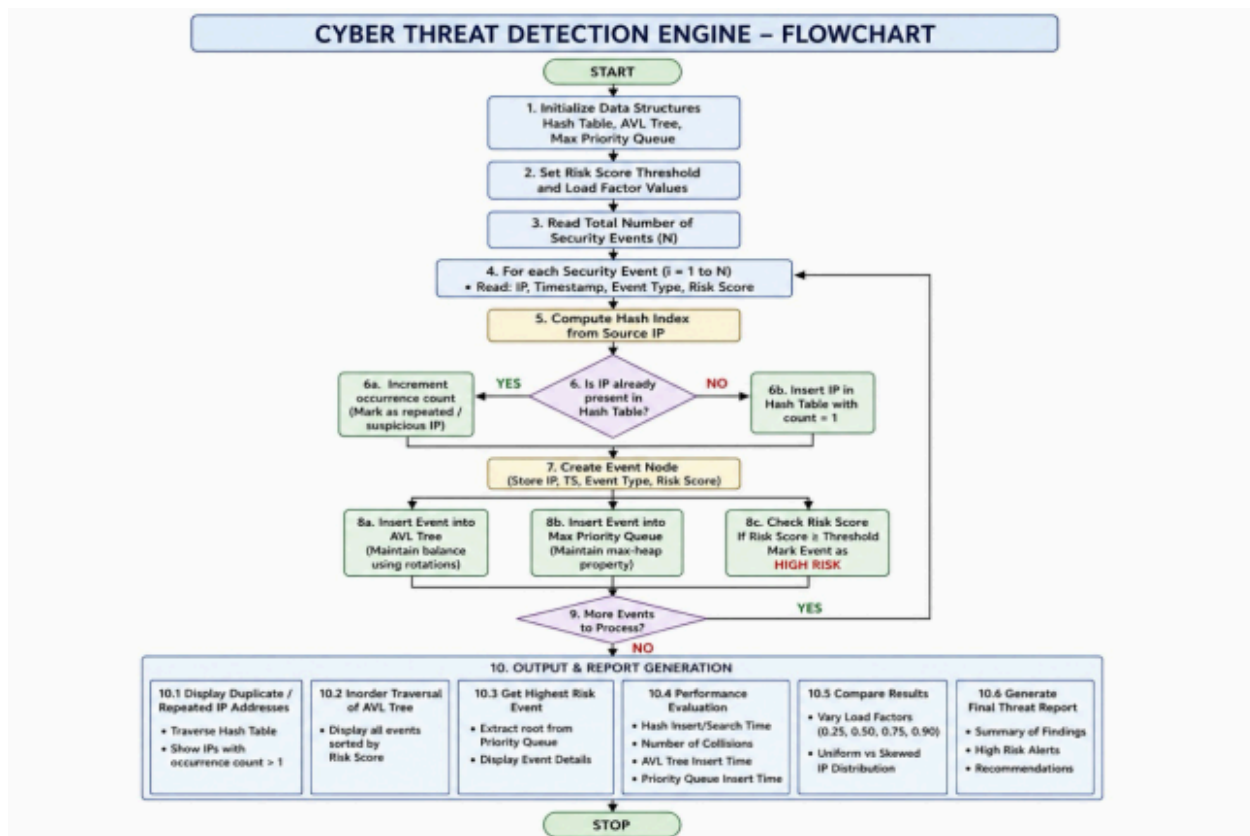
The proposed system consists of two main data structures: **Hash Table and AVL Tree**. When a network indicator is received, the system first validates the input and searches the hash table. If the indicator is present, the system immediately generates a threat alert. If required, the AVL tree is used to maintain and search threat information in balanced order.

Basic architecture:



Hashing provides average **O(1)** lookup, while AVL tree operations take **O(log n)** time.

5. Algorithm / Pseudocode / Flowchart



Pseudocode:

START

Create Hash Table

Create AVL Tree

Add known cyber threats

Read incoming threat indicator

Search indicator in Hash Table

IF indicator is found

Display "CYBER THREAT DETECTED"

ELSE

Search indicator in AVL Tree

IF indicator is found

Display "CYBER THREAT DETECTED"

ELSE

Display "NO KNOWN THREAT"

END IF

END IF

STOP

AVL Tree Insertion:

START

Read new threat

Insert threat according to BST rules

Calculate balance factor

IF tree becomes unbalanced

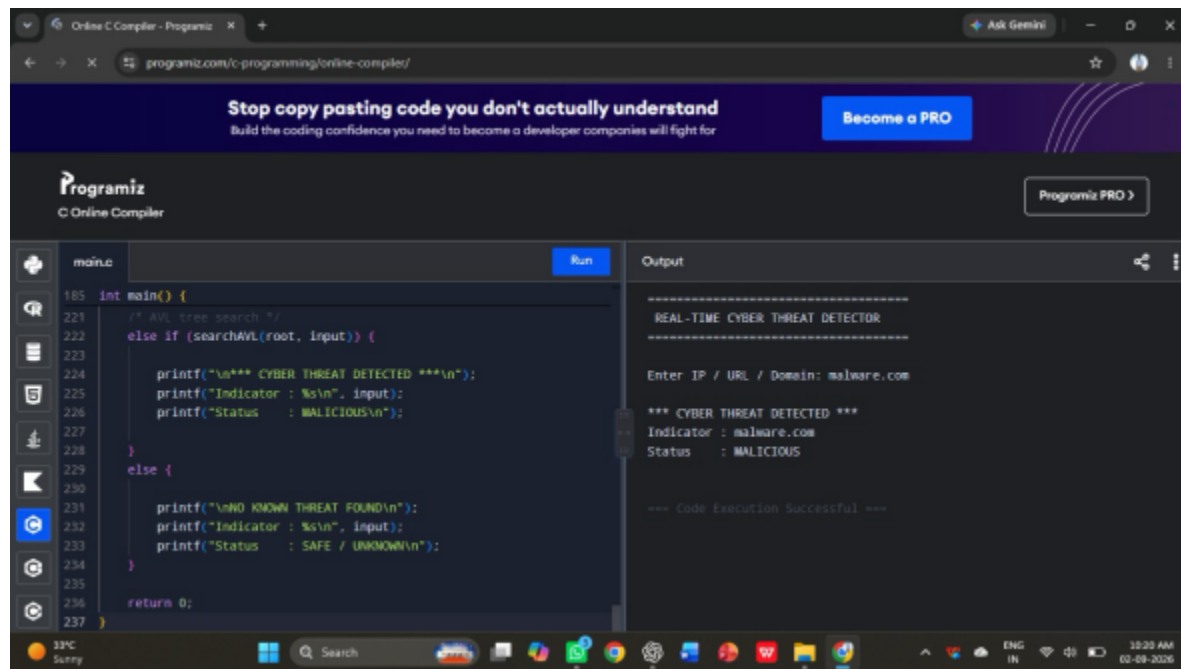
Perform LL, RR, LR or RL rotation

Update node height

Return balanced tree

STOP

OUT PUT :FIG 1



6. Implementation / Source Code

C-Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#define SIZE 10
```

```
/* ----- AVL TREE ----- */
```

```
struct Node {
    char threat[50];
    int height;
    struct Node *left, *right;
};
```

```
int height(struct Node *n) {
    if (n == NULL)
        return 0;
    return n->height;
}
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
struct Node* createNode(char threat[]) {  
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));  
  
    strcpy(node->threat, threat);  
    node->height = 1;  
    node->left = node->right = NULL;  
  
    return node;  
}
```

```
int balanceFactor(struct Node *n) {  
    if (n == NULL)  
        return 0;  
    return height(n->left) - height(n->right);  
}
```

```
struct Node* rightRotate(struct Node *y) {  
    struct Node *x = y->left;  
    struct Node *temp = x->right;  
  
    x->right = y;  
    y->left = temp;  
  
    y->height = 1 + max(height(y->left), height(y->right));  
    x->height = 1 + max(height(x->left), height(x->right));  
    return x;  
}
```

```
struct Node* leftRotate(struct Node *x) {  
    struct Node *y = x->right;  
    struct Node *temp = y->left;  
  
    y->left = x;  
    x->right = temp;  
  
    x->height = 1 + max(height(x->left), height(x->right));  
    y->height = 1 + max(height(y->left), height(y->right));  
}
```



```

    return y;
}

struct Node* insertAVL(struct Node *root, char threat[]) {

    if (root == NULL)
        return createNode(threat);

    if (strcmp(threat, root->threat) < 0)
        root->left = insertAVL(root->left, threat);

    else if (strcmp(threat, root->threat) > 0)
        root->right = insertAVL(root->right, threat);

    else
        return root;

    root->height = 1 + max(height(root->left),
                          height(root->right));

    int balance = balanceFactor(root);

    /* LL Case */
    if (balance > 1 &&
        strcmp(threat, root->left->threat) < 0)
        return rightRotate(root);

    /* RR Case */
    if (balance < -1 &&
        strcmp(threat, root->right->threat) > 0)
        return leftRotate(root);

    /* LR Case */
    if (balance > 1 &&
        strcmp(threat, root->left->threat) > 0) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }

    /* RL Case */

```

```

    if (balance < -1 &&
        strcmp(threat, root->right->threat) < 0) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }

    return root;
}
int searchAVL(struct Node *root, char threat[]) {

    if (root == NULL)
        return 0;

    if (strcmp(threat, root->threat) == 0)
        return 1;

    if (strcmp(threat, root->threat) < 0)
        return searchAVL(root->left, threat);

    return searchAVL(root->right, threat);
}

/* ----- HASH TABLE ----- */

char hashTable[SIZE][50];
int occupied[SIZE] = {0};

int hashFunction(char threat[]) {
    unsigned long hash = 0;
    int i;

    for (i = 0; threat[i] != '\0'; i++)
        hash = (hash * 31 + threat[i]) % SIZE;

    return hash;
}

void insertHash(char threat[]) {

    int index = hashFunction(threat);
    int start = index;

```

```

while (occupied[index]) {

    if (strcmp(hashTable[index], threat) == 0)
        return;

    index = (index + 1) % SIZE;
    if (index == start) {
        printf("Hash table is full!\n");
        return;
    }
}

strcpy(hashTable[index], threat);
occupied[index] = 1;
}

int searchHash(char threat[]) {

    int index = hashFunction(threat);
    int start = index;

    while (occupied[index]) {

        if (strcmp(hashTable[index], threat) == 0)
            return 1;

        index = (index + 1) % SIZE;

        if (index == start)
            break;
    }

    return 0;
}

/* ----- MAIN PROGRAM ----- */

int main() {

    struct Node *root = NULL;

```

```

char threats[][50] = {
    "192.168.1.10",
    "10.10.10.50",
    "malware.com",
    "badsite.net"
};
int n = 4;
int i;
char input[50];

/* Store threats */
for (i = 0; i < n; i++) {
    insertHash(threats[i]);
    root = insertAVL(root, threats[i]);
}

printf("=====\n");
printf(" REAL-TIME CYBER THREAT DETECTOR\n");
printf("=====\n");

printf("\nEnter IP / URL / Domain: ");
scanf("%49s", input);

/* Hash table search */
if (searchHash(input)) {

    printf("\n*** CYBER THREAT DETECTED ***\n");
    printf("Indicator : %s\n", input);
    printf("Status   : MALICIOUS\n");

}

/* AVL tree search */
else if (searchAVL(root, input)) {

    printf("\n*** CYBER THREAT DETECTED ***\n");
    printf("Indicator : %s\n", input);
    printf("Status   : MALICIOUS\n");

}

else {

```

```

        printf("\nNO KNOWN THREAT FOUND\n");
        printf("Indicator : %s\n", input);
        printf("Status   : SAFE / UNKNOWN\n");
    }

    return 0;
}

```

7. Test Cases and Expected/Actual Results

Test Case	Input	Expected Result	Actual Result
TC01	192.168.1.1 0	Threat Detected	Threat Detected
TC02	malware.co m	Threat Detected	Threat Detected
TC03	10.10.10.50	Threat Detected	Threat Detected
TC04	google.com	No Known Threat	No Known Threat
TC05	badsite.net	Threat Detected	Threat Detected

The test cases demonstrate that known malicious indicators are detected while unknown indicators are not reported as known threats.

8. Execution Screenshots / Output

Sample Output 1:

```

Enter threat indicator: 192.168.1.10
CYBER THREAT DETECTED
Severity: High

```

Sample Output 2:

```

Enter threat indicator: malware.com
CYBER THREAT DETECTED
Severity: Critical

```

Sample Output 3:

```

Enter threat indicator: google.com
NO KNOWN THREAT

```

For the final report, screenshots of the Python program execution can be inserted under this section.

9. Analysis and Discussion

The system demonstrates that the selection of an appropriate data structure significantly affects performance. Linear search requires $O(n)$ time, whereas hashing provides an average $O(1)$ search operation. The AVL tree provides $O(\log n)$ search, insertion and deletion while keeping the tree balanced.

Method	Average Search	Main Advantage
Linear Search	$O(n)$	Simple
Hash Table	$O(1)$	Very fast lookup
AVL Tree	$O(\log n)$	Balanced and ordered data

Hashing is suitable for fast exact threat identification, while the AVL tree is useful when organized and balanced data storage is required. Therefore, using both structures provides a flexible solution for cyber threat detection.

10. Conclusion

The **Real-Time Cyber Threat Detection Engine** successfully demonstrates the application of hashing and self-balancing trees to cybersecurity. Hashing provides rapid lookup of known malicious indicators, while the AVL tree maintains balanced data and supports efficient searching. The proposed system can be extended with real-time network packet monitoring, external threat intelligence databases, graphical dashboards and machine-learning-based anomaly detection. Overall, the project shows how data structures can be applied to solve practical cybersecurity problems.

11. Individual Contribution of Group Members

Group Member	Contribution
Member 1	Problem analysis, requirement collection and Hash Table implementation
Member 2	AVL Tree implementation, algorithms and testing
Member 3	Integration, output testing, documentation and report preparation

If your group has only 2 members, the third row can be removed.

12. References

1. Thomas H. Cormen et al., *Introduction to Algorithms*, MIT Press.
2. Mark Allen Weiss, *Data Structures and Algorithm Analysis in Python*, Pearson.
3. Python Documentation, *Python 3 Documentation*.
4. William Stallings, *Computer Security: Principles and Practice*, Pearson.
5. National Institute of Standards and Technology (NIST), *Cybersecurity Framework*.
6. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, **Introduction to Algorithms**, MIT Press.
7. Mark Allen Weiss, **Data Structures and Algorithm Analysis in Python**, Pearson.
8. William Stallings, **Computer Security: Principles and Practice**, Pearson